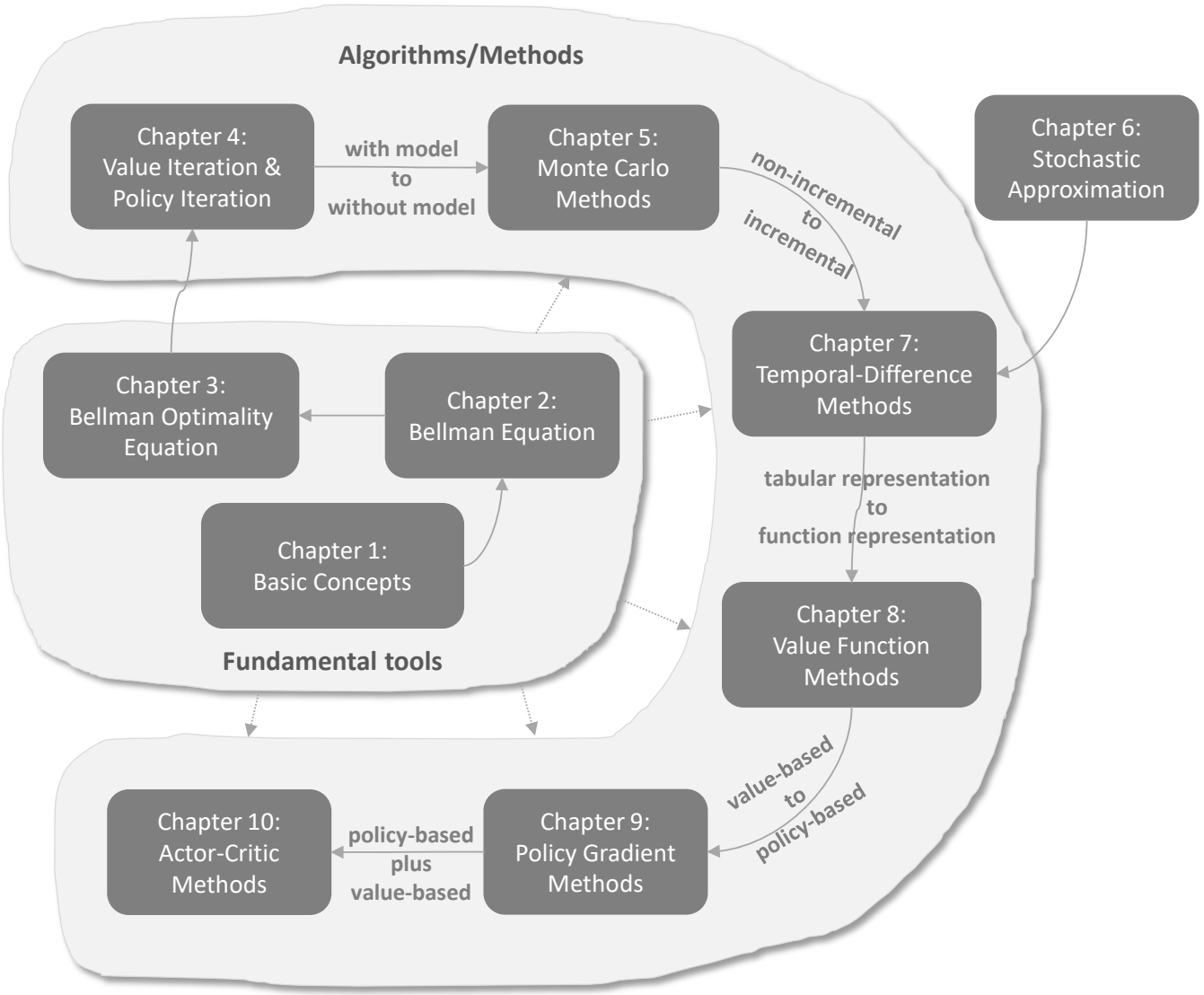
第四讲：值迭代与策略迭代

April 11, 2026

大纲


Algorithms/Methods
Chapter 4: Value Iteration & Policy Iteration
with model to without model
Chapter 5: Monte Carlo Methods
non-incremental to incremental
Chapter 6: Stochastic Approximation
Chapter 7: Temporal-Difference Methods
tabular representation to function representation
Chapter 8: Value Function Methods
value-based to policy-based
Chapter 9: Policy Gradient Methods
policy-based plus value-based
Chapter 10: Actor-Critic Methods
Chapter 3: Bellman Optimality Equation
Chapter 2: Bellman Equation
Chapter 1: Basic Concepts
Fundamental tools

大纲

1 值迭代算法

2 策略迭代算法

3 截断策略迭代算法

大纲

1 值迭代算法

2 策略迭代算法

3 截断策略迭代算法

值迭代算法

- 如何求解贝尔曼最优方程?

$$v = f(v) = \max_{\pi}(r_\pi + \gamma P_\pi v)$$

- **压缩映射定理**提示了一种迭代算法:

$$v_{k+1} = f(v_k) = \max_{\pi}(r_\pi + \gamma P_\pi v_k), \quad k = 0, 1, 2, 3 \ldots$$

其中 v_0 可以任意选取。 该算法最终可以找到最优状态价值和一个最优策略。

- 该算法称为值迭代算法!
- 接下来我们研究该算法的**实现细节**。

值迭代算法

▷ 如何求解贝尔曼最优方程?

$$v = f(v) = \max_{\pi}(r_\pi + \gamma P_\pi v)$$

▷ **压缩映射定理**提示了一种迭代算法:

$$v_{k+1} = f(v_k) = \max_{\pi}(r_\pi + \gamma P_\pi v_k), \quad k = 0, 1, 2, 3 \ldots$$

其中 v_0 可以任意选取。 该算法最终可以找到最优状态价值和一个最优策略。

▷ 该算法称为值迭代算法!

▷ 接下来我们研究该算法的**实现细节**。

值迭代算法

▷ 如何求解贝尔曼最优方程?

$$v = f(v) = \max_{\pi}(r_\pi + \gamma P_\pi v)$$

▷ **压缩映射定理**提示了一种迭代算法:

$$v_{k+1} = f(v_k) = \max_{\pi}(r_\pi + \gamma P_\pi v_k), \quad k = 0, 1, 2, 3 \ldots$$

其中 v_0 可以任意选取。 该算法最终可以找到最优状态价值和一个最优策略。

▷ 该算法称为值迭代算法!

▷ 接下来我们研究该算法的**实现细节**。

值迭代算法

算法（矩阵-向量形式）

$$v_{k+1} = f(v_k) = \max_{\pi}(r_\pi + \gamma P_\pi v_k), \quad k = 0, 1, 2, 3 \ldots$$

可以分解为两个步骤。

- 步骤一：策略更新。该步骤求解

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_k)$$

  其中 v_k 为已知量。

- 步骤二：价值更新。该步骤计算：

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$$

问题： v_k 是状态价值吗？

答案： 不是，因为 v_k 不一定满足任何贝尔曼方程。

值迭代算法

算法（矩阵-向量形式）

$$v_{k+1} = f(v_k) = \max_{\pi}(r_\pi + \gamma P_\pi v_k), \quad k = 0, 1, 2, 3 \ldots$$

可以分解为两个步骤。

- 步骤一：策略更新。该步骤求解

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_k)$$

  其中 v_k 为已知量。

- 步骤二：价值更新。该步骤计算：

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$$

问题： v_k 是状态价值吗？

答案： 不是，因为 v_k 不一定满足任何贝尔曼方程。

值迭代算法

算法（矩阵-向量形式）

$$v_{k+1} = f(v_k) = \max_{\pi}(r_\pi + \gamma P_\pi v_k), \quad k = 0, 1, 2, 3 \ldots$$

可以分解为两个步骤。

- 步骤一：策略更新。该步骤求解

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_k)$$

  其中 v_k 为已知量。

- 步骤二：价值更新。该步骤计算：

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$$

问题： v_k 是状态价值吗？

答案： 不是，因为 v_k 不一定满足任何贝尔曼方程。

值迭代算法

算法（矩阵-向量形式）

$$v_{k+1} = f(v_k) = \max_{\pi}(r_\pi + \gamma P_\pi v_k), \quad k = 0, 1, 2, 3 \ldots$$

可以分解为两个步骤。

- 步骤一：策略更新。该步骤求解

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_k)$$

  其中 v_k 为已知量。

- 步骤二：价值更新。该步骤计算：

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$$

问题：v_k 是状态价值吗?

答案：不是，因为 v_k 不一定满足任何贝尔曼方程。

值迭代算法

▷ 接下来研究逐元素形式，以便实现该算法。

- 矩阵-向量形式适用于**理论分析**。
- 逐元素形式适用于**实现**。

值迭代算法——逐元素形式

▷ 步骤一：策略更新

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_k)$$

的逐元素形式为

$$\pi_{k+1}(s) = \arg\max_{\pi} \sum_a \pi(a|s) \underbrace{\left(\sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_k(s') \right)}_{q_k(s,a)}, \quad s \in \mathcal{S}$$

求解上述优化问题的最优策略为

$$\pi_{k+1}(a|s) = \begin{cases} 1 & a = a_k^*(s) \\ 0 & a \neq a_k^*(s) \end{cases}$$

其中 $a_k^*(s) = \arg\max_a q_k(a,s)$。$\pi_{k+1}$ 称为**贪婪策略**，因为它直接选择最大的 q 值所对应的动作。

值迭代算法——逐元素形式

▷ 步骤一：策略更新

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_k)$$

的逐元素形式为

$$\pi_{k+1}(s) = \arg\max_{\pi} \sum_a \pi(a|s) \underbrace{\left(\sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_k(s') \right)}_{q_k(s,a)}, \quad s \in \mathcal{S}$$

求解上述优化问题的最优策略为

$$\pi_{k+1}(a|s) = \begin{cases} 1 & a = a_k^*(s) \\ 0 & a \neq a_k^*(s) \end{cases}$$

其中 $a_k^*(s) = \arg\max_a q_k(a,s)$。 π_{k+1} 称为**贪婪策略**，因为它直接选择最大的 q 值所对应的动作。

值迭代算法——逐元素形式

▷ 步骤二：价值更新

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$$

的逐元素形式为

$$v_{k+1}(s) = \sum_a \pi_{k+1}(a|s) \underbrace{\left(\sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_k(s') \right)}_{q_k(s,a)}, \quad s \in \mathcal{S}$$

由于 π_{k+1} 是贪婪策略，上式简化为

$$v_{k+1}(s) = \max_a q_k(a,s)$$

值迭代算法——逐元素形式

▷ 步骤二：价值更新

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$$

的逐元素形式为

$$v_{k+1}(s) = \sum_a \pi_{k+1}(a|s) \underbrace{\left(\sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_k(s') \right)}_{q_k(s,a)}, \quad s \in \mathcal{S}$$

由于 π_{k+1} 是贪婪策略，上式简化为

$$v_{k+1}(s) = \max_a q_k(a,s)$$

值迭代算法——伪代码

▷ 流程总结:

$$v_k(s) \to q_k(s,a) \to \text{贪婪策略 } \pi_{k+1}(a|s) \to \text{新价值 } v_{k+1} = \max_a q_k(s,a)$$

值迭代算法——伪代码

▷ 流程总结：

$$v_k(s) \to q_k(s,a) \to \text{贪婪策略 } \pi_{k+1}(a|s) \to \text{新价值 } v_{k+1} = \max_a q_k(s,a)$$

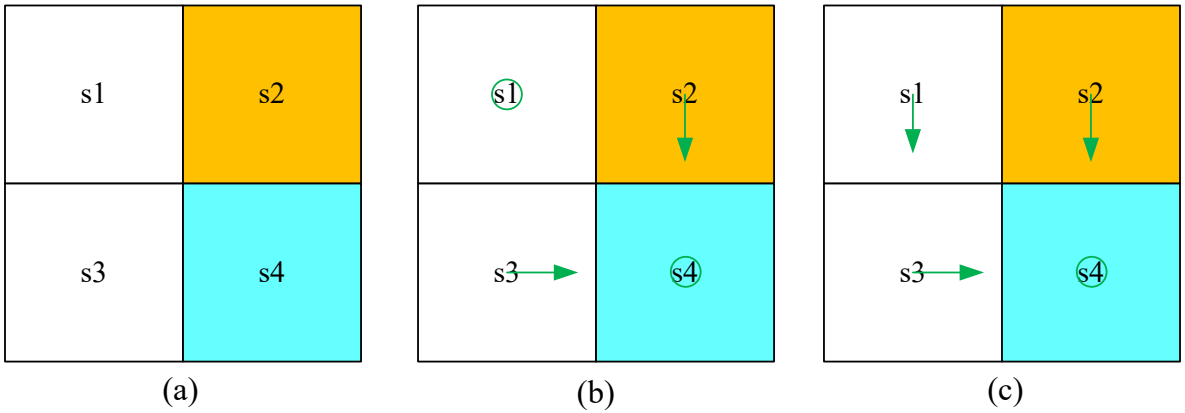
伪代码：值迭代算法

初始化： 所有 (s,a) 的概率模型 $p(r|s,a)$ 和 $p(s'|s,a)$ 已知。初始值 v_0。
目标： 通过求解贝尔曼最优方程，搜索最优状态价值和最优策略。

当 v_k 尚未收敛（即 $\|v_k - v_{k-1}\|$ 大于预设小阈值）时，执行第 k 次迭代：
对每个状态 $s \in \mathcal{S}$，执行：
对每个动作 $a \in \mathcal{A}(s)$，执行：
q 值：$q_k(s,a) = \sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a)v_k(s')$
最大动作价值：$a_k^*(s) = \arg\max_a q_k(a,s)$
策略更新：若 $a = a_k^*$ 则 $\pi_{k+1}(a|s) = 1$，否则 $\pi_{k+1}(a|s) = 0$
价值更新：$v_{k+1}(s) = \max_a q_k(a,s)$

值迭代算法——示例

▷ 奖励设置：$r_{\text{boundary}} = r_{\text{forbidden}} = -1$，$r_{\text{target}} = 1$，折扣因子 $\gamma = 0.9$。

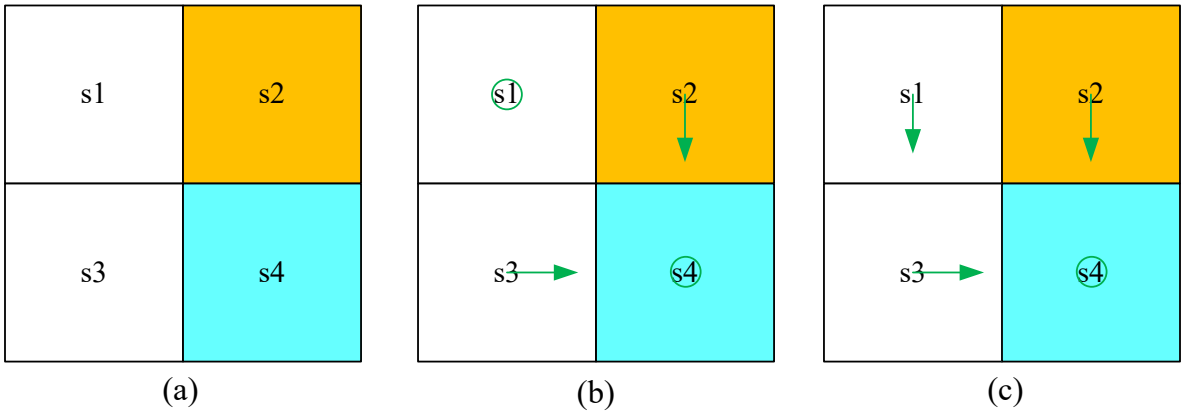


q 表：$q(s,a) = \sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a)v_k(s')$ 的表达式：

q 值	a_1	a_2	a_3	a_4	a_5
s_1	$-1+\gamma v(s_1)$	$-1+\gamma v(s_2)$	$0+\gamma v(s_3)$	$-1+\gamma v(s_1)$	$0+\gamma v(s_1)$
s_2	$-1+\gamma v(s_2)$	$-1+\gamma v(s_2)$	$1+\gamma v(s_4)$	$0+\gamma v(s_1)$	$-1+\gamma v(s_2)$
s_3	$0+\gamma v(s_1)$	$1+\gamma v(s_4)$	$-1+\gamma v(s_3)$	$-1+\gamma v(s_3)$	$0+\gamma v(s_3)$
s_4	$-1+\gamma v(s_2)$	$-1+\gamma v(s_4)$	$-1+\gamma v(s_4)$	$0+\gamma v(s_3)$	$1+\gamma v(s_4)$

值迭代算法——示例

▷ 奖励设置：$r_{\text{boundary}} = r_{\text{forbidden}} = -1$，$r_{\text{target}} = 1$，折扣因子 $\gamma = 0.9$。



q 表：$q(s,a) = \sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_k(s')$ 的表达式：

q 值	a_1	a_2	a_3	a_4	a_5
s_1	$-1+\gamma v(s_1)$	$-1+\gamma v(s_2)$	$0+\gamma v(s_3)$	$-1+\gamma v(s_1)$	$0+\gamma v(s_1)$
s_2	$-1+\gamma v(s_2)$	$-1+\gamma v(s_2)$	$1+\gamma v(s_4)$	$0+\gamma v(s_1)$	$-1+\gamma v(s_2)$
s_3	$0+\gamma v(s_1)$	$1+\gamma v(s_4)$	$-1+\gamma v(s_3)$	$-1+\gamma v(s_3)$	$0+\gamma v(s_3)$
s_4	$-1+\gamma v(s_2)$	$-1+\gamma v(s_4)$	$-1+\gamma v(s_4)$	$0+\gamma v(s_3)$	$1+\gamma v(s_4)$

值迭代算法——示例

- $k = 0$: 令 $v_0(s_1) = v_0(s_2) = v_0(s_3) = v_0(s_4) = 0$

q 值	a_1	a_2	a_3	a_4	a_5
s_1	-1	-1	0	-1	0
s_2	-1	-1	1	0	-1
s_3	0	1	-1	-1	0
s_4	-1	-1	-1	0	1

步骤一：策略更新：

$$\pi_1(a_5|s_1) = 1, \quad \pi_1(a_3|s_2) = 1, \quad \pi_1(a_2|s_3) = 1, \quad \pi_1(a_5|s_4) = 1$$

步骤二：价值更新：

$$v_1(s_1) = 0, \quad v_1(s_2) = 1, \quad v_1(s_3) = 1, \quad v_1(s_4) = 1$$

该策略如图 (b) 所示。

值迭代算法——示例

- $k = 0$: 令 $v_0(s_1) = v_0(s_2) = v_0(s_3) = v_0(s_4) = 0$

q 值	a_1	a_2	a_3	a_4	a_5
s_1	-1	-1	0	-1	0
s_2	-1	-1	1	0	-1
s_3	0	1	-1	-1	0
s_4	-1	-1	-1	0	1

步骤一：策略更新：

$$\pi_1(a_5|s_1) = 1, \quad \pi_1(a_3|s_2) = 1, \quad \pi_1(a_2|s_3) = 1, \quad \pi_1(a_5|s_4) = 1$$

步骤二：价值更新：

$$v_1(s_1) = 0, \quad v_1(s_2) = 1, \quad v_1(s_3) = 1, \quad v_1(s_4) = 1$$

该策略如图 (b) 所示。

值迭代算法——示例

- $k = 0$: 令 $v_0(s_1) = v_0(s_2) = v_0(s_3) = v_0(s_4) = 0$

q 值	a_1	a_2	a_3	a_4	a_5
s_1	-1	-1	0	-1	0
s_2	-1	-1	1	0	-1
s_3	0	1	-1	-1	0
s_4	-1	-1	-1	0	1

步骤一：策略更新：

$$\pi_1(a_5|s_1) = 1, \quad \pi_1(a_3|s_2) = 1, \quad \pi_1(a_2|s_3) = 1, \quad \pi_1(a_5|s_4) = 1$$

步骤二：价值更新：

$$v_1(s_1) = 0, \quad v_1(s_2) = 1, \quad v_1(s_3) = 1, \quad v_1(s_4) = 1$$

该策略如图 (b) 所示。

值迭代算法——示例

- $k=1$：由于 $v_1(s_1)=0, v_1(s_2)=1, v_1(s_3)=1, v_1(s_4)=1$，可得

q 表	a_1	a_2	a_3	a_4	a_5
s_1	$-1+\gamma 0$	$-1+\gamma 1$	$0+\gamma 1$	$-1+\gamma 0$	$0+\gamma 0$
s_2	$-1+\gamma 1$	$-1+\gamma 1$	$1+\gamma 1$	$0+\gamma 0$	$-1+\gamma 1$
s_3	$0+\gamma 0$	$1+\gamma 1$	$-1+\gamma 1$	$-1+\gamma 1$	$0+\gamma 1$
s_4	$-1+\gamma 1$	$-1+\gamma 1$	$-1+\gamma 1$	$0+\gamma 1$	$1+\gamma 1$

步骤一：策略更新：

$$\pi_2(a_3|s_1)=1,\ \pi_2(a_3|s_2)=1,\ \pi_2(a_2|s_3)=1,\ \pi_2(a_5|s_4)=1.$$

步骤二：价值更新：

$$v_2(s_1)=\gamma 1,\ v_2(s_2)=1+\gamma 1,\ v_2(s_3)=1+\gamma 1,\ v_2(s_4)=1+\gamma 1.$$

该策略如图 (c) 所示，已达到最优！！

- $k=2,3,\ldots$。当 $\|v_k-v_{k+1}\|$ 小于预设阈值时停止。

值迭代算法——示例

- $k=1$：由于 $v_1(s_1)=0, v_1(s_2)=1, v_1(s_3)=1, v_1(s_4)=1$，可得

q 表	a_1	a_2	a_3	a_4	a_5
s_1	$-1+\gamma 0$	$-1+\gamma 1$	$0+\gamma 1$	$-1+\gamma 0$	$0+\gamma 0$
s_2	$-1+\gamma 1$	$-1+\gamma 1$	$1+\gamma 1$	$0+\gamma 0$	$-1+\gamma 1$
s_3	$0+\gamma 0$	$1+\gamma 1$	$-1+\gamma 1$	$-1+\gamma 1$	$0+\gamma 1$
s_4	$-1+\gamma 1$	$-1+\gamma 1$	$-1+\gamma 1$	$0+\gamma 1$	$1+\gamma 1$

步骤一：策略更新：

$$\pi_2(a_3|s_1)=1,\ \pi_2(a_3|s_2)=1,\ \pi_2(a_2|s_3)=1,\ \pi_2(a_5|s_4)=1.$$

步骤二：价值更新：

$$v_2(s_1)=\gamma 1,\ v_2(s_2)=1+\gamma 1,\ v_2(s_3)=1+\gamma 1,\ v_2(s_4)=1+\gamma 1.$$

该策略如图 (c) 所示，已达到最优！！

- $k=2,3,\ldots$。当 $\|v_k-v_{k+1}\|$ 小于预设阈值时停止。

值迭代算法——示例

- $k=1$：由于 $v_1(s_1)=0, v_1(s_2)=1, v_1(s_3)=1, v_1(s_4)=1$，可得

q 表	a_1	a_2	a_3	a_4	a_5
s_1	$-1+\gamma 0$	$-1+\gamma 1$	$0+\gamma 1$	$-1+\gamma 0$	$0+\gamma 0$
s_2	$-1+\gamma 1$	$-1+\gamma 1$	$1+\gamma 1$	$0+\gamma 0$	$-1+\gamma 1$
s_3	$0+\gamma 0$	$1+\gamma 1$	$-1+\gamma 1$	$-1+\gamma 1$	$0+\gamma 1$
s_4	$-1+\gamma 1$	$-1+\gamma 1$	$-1+\gamma 1$	$0+\gamma 1$	$1+\gamma 1$

步骤一：策略更新：

$$\pi_2(a_3|s_1)=1,\ \pi_2(a_3|s_2)=1,\ \pi_2(a_2|s_3)=1,\ \pi_2(a_5|s_4)=1.$$

步骤二：价值更新：

$$v_2(s_1)=\gamma 1,\ v_2(s_2)=1+\gamma 1,\ v_2(s_3)=1+\gamma 1,\ v_2(s_4)=1+\gamma 1.$$

该策略如图 (c) 所示，已达到最优！！

- $k=2,3,\ldots$。当 $\|v_k-v_{k+1}\|$ 小于预设阈值时停止。

值迭代算法——示例

- $k=1$：由于 $v_1(s_1)=0, v_1(s_2)=1, v_1(s_3)=1, v_1(s_4)=1$，可得

q 表	a_1	a_2	a_3	a_4	a_5
s_1	$-1+\gamma 0$	$-1+\gamma 1$	$0+\gamma 1$	$-1+\gamma 0$	$0+\gamma 0$
s_2	$-1+\gamma 1$	$-1+\gamma 1$	$1+\gamma 1$	$0+\gamma 0$	$-1+\gamma 1$
s_3	$0+\gamma 0$	$1+\gamma 1$	$-1+\gamma 1$	$-1+\gamma 1$	$0+\gamma 1$
s_4	$-1+\gamma 1$	$-1+\gamma 1$	$-1+\gamma 1$	$0+\gamma 1$	$1+\gamma 1$

步骤一：策略更新：

$$\pi_2(a_3|s_1)=1,\ \pi_2(a_3|s_2)=1,\ \pi_2(a_2|s_3)=1,\ \pi_2(a_5|s_4)=1.$$

步骤二：价值更新：

$$v_2(s_1)=\gamma 1,\ v_2(s_2)=1+\gamma 1,\ v_2(s_3)=1+\gamma 1,\ v_2(s_4)=1+\gamma 1.$$

该策略如图 (c) 所示，已达到最优！！

- $k=2,3,\ldots$。当 $\|v_k-v_{k+1}\|$ 小于预设阈值时停止。

值迭代算法——示例

- $k=1$：由于 $v_1(s_1)=0, v_1(s_2)=1, v_1(s_3)=1, v_1(s_4)=1$，可得

q 表	a_1	a_2	a_3	a_4	a_5
s_1	$-1+\gamma 0$	$-1+\gamma 1$	$0+\gamma 1$	$-1+\gamma 0$	$0+\gamma 0$
s_2	$-1+\gamma 1$	$-1+\gamma 1$	$1+\gamma 1$	$0+\gamma 0$	$-1+\gamma 1$
s_3	$0+\gamma 0$	$1+\gamma 1$	$-1+\gamma 1$	$-1+\gamma 1$	$0+\gamma 1$
s_4	$-1+\gamma 1$	$-1+\gamma 1$	$-1+\gamma 1$	$0+\gamma 1$	$1+\gamma 1$

步骤一：策略更新：

$$\pi_2(a_3|s_1)=1,\ \pi_2(a_3|s_2)=1,\ \pi_2(a_2|s_3)=1,\ \pi_2(a_5|s_4)=1.$$

步骤二：价值更新：

$$v_2(s_1)=\gamma 1,\ v_2(s_2)=1+\gamma 1,\ v_2(s_3)=1+\gamma 1,\ v_2(s_4)=1+\gamma 1.$$

该策略如图 (c) 所示，已达到最优！！

- $k=2,3,\ldots$。当 $\|v_k-v_{k+1}\|$ 小于预设阈值时停止。

大纲

1 值迭代算法

2 策略迭代算法

3 截断策略迭代算法

策略迭代算法

▷ 算法描述:

给定一个随机的初始策略 π_0,

- 步骤一: 策略评估（PE）

  该步骤计算 π_k 的状态价值:

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

  注意, v_{π_k} 是状态价值函数。

- 步骤二: 策略改进（PI）

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_{\pi_k})$$

  最大化按分量进行!

与值迭代算法类似? 请耐心等待, 稍后将进行比较。

策略迭代算法

▷ 算法描述:

给定一个随机的初始策略 π_0,

- 步骤一: 策略评估 (PE)

  该步骤计算 π_k 的状态价值:

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

  注意, v_{π_k} 是状态价值函数。

- 步骤二: 策略改进 (PI)

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_{\pi_k})$$

  最大化按分量进行!

与值迭代算法类似? 请耐心等待, 稍后将进行比较。

策略迭代算法

▷ 算法描述:

给定一个随机的初始策略 π_0,

- 步骤一：策略评估（PE）

  该步骤计算 π_k 的状态价值:

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

  注意，v_{π_k} 是状态价值函数。

- 步骤二：策略改进（PI）

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_{\pi_k})$$

  最大化按分量进行！

与值迭代算法类似？请耐心等待，稍后将进行比较。

策略迭代算法

▷ 算法描述:

给定一个随机的初始策略 π_0,

- 步骤一: 策略评估 (PE)

  该步骤计算 π_k 的状态价值:

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

  注意, v_{π_k} 是状态价值函数。

- 步骤二: 策略改进 (PI)

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_{\pi_k})$$

  最大化按分量进行!

与值迭代算法类似? 请耐心等待, 稍后将进行比较。

策略迭代算法

▷ 该算法产生如下序列:

$$\pi_0 \xrightarrow{PE} v_{\pi_0} \xrightarrow{PI} \pi_1 \xrightarrow{PE} v_{\pi_1} \xrightarrow{PI} \pi_2 \xrightarrow{PE} v_{\pi_2} \xrightarrow{PI} \dots$$

PE=策略评估，PI=策略改进

▷ 问题:

- 问题一: 在策略评估步骤中，如何通过求解贝尔曼方程得到状态价值 v_{π_k}?
- 问题二: 在策略改进步骤中，为什么新策略 π_{k+1} 优于 π_k?
- 问题三: 为什么这样的迭代算法最终能找到最优策略?
- 问题四: 该策略迭代算法与值迭代算法有何关系?

策略迭代算法

▷ 该算法产生如下序列：

$$\pi_0 \xrightarrow{PE} v_{\pi_0} \xrightarrow{PI} \pi_1 \xrightarrow{PE} v_{\pi_1} \xrightarrow{PI} \pi_2 \xrightarrow{PE} v_{\pi_2} \xrightarrow{PI} \dots$$

PE=策略评估，PI=策略改进

▷ 问题：

- 问题一：在策略评估步骤中，如何通过求解贝尔曼方程得到状态价值 v_{π_k}？
- 问题二：在策略改进步骤中，为什么新策略 π_{k+1} 优于 π_k？
- 问题三：为什么这样的迭代算法最终能找到最优策略？
- 问题四：该策略迭代算法与值迭代算法有何关系？

策略迭代算法

▷ 该算法产生如下序列：

$$\pi_0 \xrightarrow{PE} v_{\pi_0} \xrightarrow{PI} \pi_1 \xrightarrow{PE} v_{\pi_1} \xrightarrow{PI} \pi_2 \xrightarrow{PE} v_{\pi_2} \xrightarrow{PI} \dots$$

PE=策略评估，PI=策略改进

▷ 问题：

- 问题一：在策略评估步骤中，如何通过求解贝尔曼方程得到状态价值 v_{π_k}？
- 问题二：在策略改进步骤中，为什么新策略 π_{k+1} 优于 π_k？
- 问题三：为什么这样的迭代算法最终能找到最优策略？
- 问题四：该策略迭代算法与值迭代算法有何关系？

策略迭代算法

▷ 该算法产生如下序列：

$$\pi_0 \xrightarrow{PE} v_{\pi_0} \xrightarrow{PI} \pi_1 \xrightarrow{PE} v_{\pi_1} \xrightarrow{PI} \pi_2 \xrightarrow{PE} v_{\pi_2} \xrightarrow{PI} \dots$$

PE=策略评估，PI=策略改进

▷ 问题：

- 问题一：在策略评估步骤中，如何通过求解贝尔曼方程得到状态价值 v_{π_k}？
- 问题二：在策略改进步骤中，为什么新策略 π_{k+1} 优于 π_k？
- 问题三：为什么这样的迭代算法最终能找到最优策略？
- 问题四：该策略迭代算法与值迭代算法有何关系？

策略迭代算法

▷ 该算法产生如下序列：

$$\pi_0 \xrightarrow{PE} v_{\pi_0} \xrightarrow{PI} \pi_1 \xrightarrow{PE} v_{\pi_1} \xrightarrow{PI} \pi_2 \xrightarrow{PE} v_{\pi_2} \xrightarrow{PI} \dots$$

PE=策略评估，PI=策略改进

▷ 问题：

- 问题一：在策略评估步骤中，如何通过求解贝尔曼方程得到状态价值 v_{π_k}？
- 问题二：在策略改进步骤中，为什么新策略 π_{k+1} 优于 π_k？
- 问题三：为什么这样的迭代算法最终能找到最优策略？
- 问题四：该策略迭代算法与值迭代算法有何关系？

策略迭代算法

▷ 问题一：在策略评估步骤中，如何通过求解贝尔曼方程得到状态价值 v_{π_k}？

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

- 闭合形式解：

$$v_{\pi_k} = (I - \gamma P_{\pi_k})^{-1} r_{\pi_k}$$

- 迭代解：

$$v_{\pi_k}^{(j+1)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(j)}, \quad j = 0, 1, 2, \ldots$$

已在贝尔曼方程相关讲次中介绍。

▷ 策略迭代是一个迭代算法，其策略评估步骤中还嵌套了另一个迭代算法！

策略迭代算法

▷ 问题一：在策略评估步骤中，如何通过求解贝尔曼方程得到状态价值 v_{π_k}？

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

- 闭合形式解：

$$v_{\pi_k} = (I - \gamma P_{\pi_k})^{-1} r_{\pi_k}$$

- 迭代解：

$$v_{\pi_k}^{(j+1)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(j)}, \quad j = 0, 1, 2, \ldots$$

已在贝尔曼方程相关讲次中介绍。

▷ 策略迭代是一个迭代算法，其策略评估步骤中还嵌套了另一个迭代算法！

策略迭代算法

▷ 问题一：在策略评估步骤中，如何通过求解贝尔曼方程得到状态价值 v_{π_k}？

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

- 闭合形式解：

$$v_{\pi_k} = (I - \gamma P_{\pi_k})^{-1} r_{\pi_k}$$

- 迭代解：

$$v_{\pi_k}^{(j+1)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(j)}, \quad j = 0, 1, 2, ...$$

已在贝尔曼方程相关讲次中介绍。

▷ 策略迭代是一个迭代算法，其策略评估步骤中还嵌套了另一个迭代算法！

策略迭代算法

▷ 问题二：在策略改进步骤中，为什么新策略 π_{k+1} 优于 π_k？

证明详见教材。

策略迭代算法

▷ 问题二：在策略改进步骤中，为什么新策略 π_{k+1} 优于 π_k？

Lemma (策略改进)

若 $\pi_{k+1} = \arg\max_\pi (r_\pi + \gamma P_\pi v_{\pi_k})$，则对任意 k，有 $v_{\pi_{k+1}} \geq v_{\pi_k}$。

证明详见教材。

策略迭代算法

▷ 问题三：为什么这样的迭代算法最终能找到最优策略?

由于每次迭代都会改进策略，我们有

$$v_{\pi_0} \leq v_{\pi_1} \leq v_{\pi_2} \leq \cdots \leq v_{\pi_k} \leq \cdots \leq v^*.$$

因此，v_{π_k} 持续**递增**并将收敛。

还需证明它收敛到最优值 v^*。

证明详见教材。

策略迭代算法

▷ 问题三：为什么这样的迭代算法最终能找到最优策略?

由于每次迭代都会改进策略，我们有

$$v_{\pi_0} \leq v_{\pi_1} \leq v_{\pi_2} \leq \cdots \leq v_{\pi_k} \leq \cdots \leq v^*.$$

因此，v_{π_k} 持续**递增**并将收敛。

还需证明它收敛到最优值 v^*。

证明详见教材。

策略迭代算法

▷ 问题三：为什么这样的迭代算法最终能找到最优策略?

由于每次迭代都会改进策略，我们有

$$v_{\pi_0} \leq v_{\pi_1} \leq v_{\pi_2} \leq \cdots \leq v_{\pi_k} \leq \cdots \leq v^*.$$

因此，v_{π_k} 持续**递增**并将收敛。

还需证明它收敛到最优值 v^*。

Theorem (策略迭代的收敛性)

策略迭代算法生成的状态价值序列 $\{v_{\pi_k}\}_{k=0}^{\infty}$ 收敛到最优状态价值 v^*。因此，策略序列 $\{\pi_k\}_{k=0}^{\infty}$ 收敛到最优策略。

证明详见教材。

策略迭代算法

▷ 问题四：策略迭代算法与值迭代算法有何关系?

稍后详细说明。

策略迭代算法——逐元素形式

步骤一：策略评估

▷ 矩阵-向量形式：$v_{\pi_k}^{(j+1)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(j)}, \quad j = 0, 1, 2, \ldots$

▷ 逐元素形式：

$$v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a|s) \left(\sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}^{(j)}(s') \right), \quad s \in \mathcal{S}$$

当 j 足够大或 $\|v_{\pi_k}^{(j+1)} - v_{\pi_k}^{(j)}\|$ 足够小时停止。

策略迭代算法——逐元素形式

步骤一：策略评估

▷ 矩阵-向量形式：$v_{\pi_k}^{(j+1)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(j)}, \quad j = 0, 1, 2, \ldots$

▷ 逐元素形式：

$$v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a|s) \left(\sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}^{(j)}(s') \right), \quad s \in \mathcal{S}$$

当 j 足够大或 $\|v_{\pi_k}^{(j+1)} - v_{\pi_k}^{(j)}\|$ 足够小时停止。

策略迭代算法——逐元素形式

步骤一：策略评估

▷ 矩阵-向量形式：$v_{\pi_k}^{(j+1)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(j)}, \quad j = 0, 1, 2, \ldots$

▷ 逐元素形式：

$$v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a|s) \left(\sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}^{(j)}(s') \right), \quad s \in \mathcal{S}$$

当 j 足够大或 $\|v_{\pi_k}^{(j+1)} - v_{\pi_k}^{(j)}\|$ 足够小时停止。

策略迭代算法——逐元素形式

步骤二：策略改进

- ▷ 矩阵-向量形式：$\pi_{k+1} = \arg\max_\pi (r_\pi + \gamma P_\pi v_{\pi_k})$
- ▷ 逐元素形式

$$\pi_{k+1}(s) = \arg\max_\pi \sum_a \pi(a|s) \underbrace{\left(\sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}(s') \right)}_{q_{\pi_k}(s,a)}, \quad s \in \mathcal{S}.$$

其中，$q_{\pi_k}(s,a)$ 是策略 π_k 下的动作价值。 令

$$a_k^*(s) = \arg\max_a q_{\pi_k}(a,s)$$

则贪婪策略为

$$\pi_{k+1}(a|s) = \begin{cases} 1 & a = a_k^*(s), \\ 0 & a \neq a_k^*(s). \end{cases}$$

策略迭代算法——逐元素形式

步骤二：策略改进

▷ 矩阵-向量形式：$\pi_{k+1} = \arg\max_\pi (r_\pi + \gamma P_\pi v_{\pi_k})$

▷ 逐元素形式

$$\pi_{k+1}(s) = \arg\max_\pi \sum_a \pi(a|s) \underbrace{\left(\sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}(s') \right)}_{q_{\pi_k}(s,a)}, \quad s \in \mathcal{S}.$$

其中，$q_{\pi_k}(s,a)$ 是策略 π_k 下的动作价值。 令

$$a_k^*(s) = \arg\max_a q_{\pi_k}(a,s)$$

则贪婪策略为

$$\pi_{k+1}(a|s) = \begin{cases} 1 & a = a_k^*(s), \\ 0 & a \neq a_k^*(s). \end{cases}$$

策略迭代算法——逐元素形式

步骤二：策略改进

▷ 矩阵-向量形式：$\pi_{k+1} = \arg\max_\pi (r_\pi + \gamma P_\pi v_{\pi_k})$
▷ 逐元素形式

$$\pi_{k+1}(s) = \arg\max_\pi \sum_a \pi(a|s) \underbrace{\left(\sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}(s') \right)}_{q_{\pi_k}(s,a)}, \quad s \in \mathcal{S}.$$

其中，$q_{\pi_k}(s,a)$ 是策略 π_k 下的动作价值。 令

$$a_k^*(s) = \arg\max_a q_{\pi_k}(a,s)$$

则贪婪策略为

$$\pi_{k+1}(a|s) = \begin{cases} 1 & a = a_k^*(s), \\ 0 & a \neq a_k^*(s). \end{cases}$$

策略迭代算法——实现

伪代码：策略迭代算法

初始化：所有 (s,a) 的概率模型 $p(r|s,a)$ 和 $p(s'|s,a)$ 已知。初始策略 π_0。

目标：搜索最优状态价值和最优策略。

当 v_{π_k} 尚未收敛时，执行第 k 次迭代：

策略评估：

初始化：任意初始值 $v_{\pi_k}^{(0)}$

当 $v_{\pi_k}^{(j)}$ 尚未收敛时，执行第 j 次迭代：

对每个状态 $s \in \mathcal{S}$，执行：

$$v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a|s) \left[\sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}^{(j)}(s') \right]$$

策略改进：

对每个状态 $s \in \mathcal{S}$，执行：

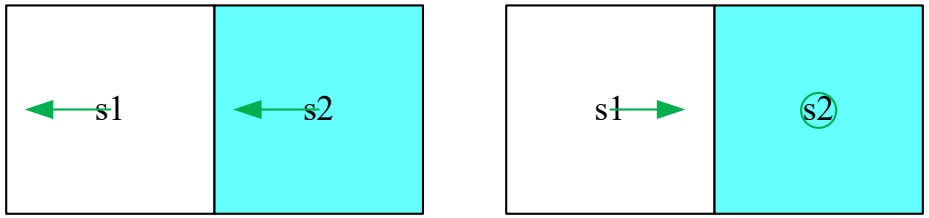
对每个动作 $a \in \mathcal{A}$，执行：

$$q_{\pi_k}(s,a) = \sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}(s')$$

$a_k^*(s) = \arg\max_a q_{\pi_k}(s,a)$

若 $a = a_k^*$ 则 $\pi_{k+1}(a|s) = 1$，否则 $\pi_{k+1}(a|s) = 0$

策略迭代算法——简单示例



(a) initial policy (b) optimal policy

- 奖励设置：$r_{\text{boundary}} = -1$，$r_{\text{target}} = 1$，折扣因子 $\gamma = 0.9$。
- 动作：a_ℓ, a_0, a_r 分别表示向左移动、保持不动和向右移动。
- 目标：使用策略迭代算法找出最优策略。

策略迭代算法——简单示例

▷ 第 $k=0$ 次迭代：步骤一：策略评估

π_0 选取为图 (a) 中的策略，贝尔曼方程为

$$v_{\pi_0}(s_1) = -1 + \gamma v_{\pi_0}(s_1),$$
$$v_{\pi_0}(s_2) = 0 + \gamma v_{\pi_0}(s_1).$$

- 直接求解方程组：

$$v_{\pi_0}(s_1) = -10, \quad v_{\pi_0}(s_2) = -9.$$

- 迭代求解，选取初始值 $v_{\pi_0}^{(0)}(s_1) = v_{\pi_0}^{(0)}(s_2) = 0$：

$$\begin{cases} v_{\pi_0}^{(1)}(s_1) = -1 + \gamma v_{\pi_0}^{(0)}(s_1) = -1, \\ v_{\pi_0}^{(1)}(s_2) = 0 + \gamma v_{\pi_0}^{(0)}(s_1) = 0, \end{cases}$$

$$\begin{cases} v_{\pi_0}^{(2)}(s_1) = -1 + \gamma v_{\pi_0}^{(1)}(s_1) = -1.9, \\ v_{\pi_0}^{(2)}(s_2) = 0 + \gamma v_{\pi_0}^{(1)}(s_1) = -0.9, \end{cases}$$

$$\begin{cases} v_{\pi_0}^{(3)}(s_1) = -1 + \gamma v_{\pi_0}^{(2)}(s_1) = -2.71, \\ v_{\pi_0}^{(3)}(s_2) = 0 + \gamma v_{\pi_0}^{(2)}(s_1) = -1.71, \end{cases}$$

$$\cdots$$

策略迭代算法——简单示例

▷ 第 $k=0$ 次迭代：步骤一：策略评估

π_0 选取为图 (a) 中的策略，贝尔曼方程为

$$v_{\pi_0}(s_1) = -1 + \gamma v_{\pi_0}(s_1),$$
$$v_{\pi_0}(s_2) = 0 + \gamma v_{\pi_0}(s_1).$$

- 直接求解方程组：

$$v_{\pi_0}(s_1) = -10, \quad v_{\pi_0}(s_2) = -9.$$

- 迭代求解，选取初始值 $v_{\pi_0}^{(0)}(s_1) = v_{\pi_0}^{(0)}(s_2) = 0$：

$$\begin{cases} v_{\pi_0}^{(1)}(s_1) = -1 + \gamma v_{\pi_0}^{(0)}(s_1) = -1, \\ v_{\pi_0}^{(1)}(s_2) = 0 + \gamma v_{\pi_0}^{(0)}(s_1) = 0, \end{cases}$$

$$\begin{cases} v_{\pi_0}^{(2)}(s_1) = -1 + \gamma v_{\pi_0}^{(1)}(s_1) = -1.9, \\ v_{\pi_0}^{(2)}(s_2) = 0 + \gamma v_{\pi_0}^{(1)}(s_1) = -0.9, \end{cases}$$

$$\begin{cases} v_{\pi_0}^{(3)}(s_1) = -1 + \gamma v_{\pi_0}^{(2)}(s_1) = -2.71, \\ v_{\pi_0}^{(3)}(s_2) = 0 + \gamma v_{\pi_0}^{(2)}(s_1) = -1.71, \end{cases}$$

$$\cdots$$

策略迭代算法——简单示例

▷ 第 $k=0$ 次迭代：步骤一：策略评估

π_0 选取为图 (a) 中的策略，贝尔曼方程为

$$v_{\pi_0}(s_1) = -1 + \gamma v_{\pi_0}(s_1),$$
$$v_{\pi_0}(s_2) = 0 + \gamma v_{\pi_0}(s_1).$$

- 直接求解方程组：

$$v_{\pi_0}(s_1) = -10, \quad v_{\pi_0}(s_2) = -9.$$

- 迭代求解，选取初始值 $v_{\pi_0}^{(0)}(s_1) = v_{\pi_0}^{(0)}(s_2) = 0$：

$$\begin{cases} v_{\pi_0}^{(1)}(s_1) = -1 + \gamma v_{\pi_0}^{(0)}(s_1) = -1, \\ v_{\pi_0}^{(1)}(s_2) = 0 + \gamma v_{\pi_0}^{(0)}(s_1) = 0, \end{cases}$$
$$\begin{cases} v_{\pi_0}^{(2)}(s_1) = -1 + \gamma v_{\pi_0}^{(1)}(s_1) = -1.9, \\ v_{\pi_0}^{(2)}(s_2) = 0 + \gamma v_{\pi_0}^{(1)}(s_1) = -0.9, \end{cases}$$
$$\begin{cases} v_{\pi_0}^{(3)}(s_1) = -1 + \gamma v_{\pi_0}^{(2)}(s_1) = -2.71, \\ v_{\pi_0}^{(3)}(s_2) = 0 + \gamma v_{\pi_0}^{(2)}(s_1) = -1.71, \end{cases}$$
$$\cdots$$

策略迭代算法——简单示例

▷ 第 $k=0$ 次迭代：步骤二：策略改进

$q_{\pi_k}(s,a)$ 的表达式：

$q_{\pi_k}(s,a)$	a_ℓ	a_0	a_r
s_1	$-1+\gamma v_{\pi_k}(s_1)$	$0+\gamma v_{\pi_k}(s_1)$	$1+\gamma v_{\pi_k}(s_2)$
s_2	$0+\gamma v_{\pi_k}(s_1)$	$1+\gamma v_{\pi_k}(s_2)$	$-1+\gamma v_{\pi_k}(s_2)$

代入 $v_{\pi_0}(s_1)=-10, v_{\pi_0}(s_2)=-9$ 和 $\gamma=0.9$，得

$q_{\pi_0}(s,a)$	a_ℓ	a_0	a_r
s_1	-10	-9	-7.1
s_2	-9	-7.1	-9.1

选取 q_{π_0} 最大值对应的动作，改进后的新策略为：

$$\pi_1(a_r|s_1)=1, \quad \pi_1(a_0|s_2)=1.$$

仅需一次迭代即达到最优策略！在编程实现中，应持续迭代直到满足停止条件。

策略迭代算法——简单示例

▷ 第 $k=0$ 次迭代：步骤二：策略改进

$q_{\pi_k}(s,a)$ 的表达式：

$q_{\pi_k}(s,a)$	a_ℓ	a_0	a_r
s_1	$-1+\gamma v_{\pi_k}(s_1)$	$0+\gamma v_{\pi_k}(s_1)$	$1+\gamma v_{\pi_k}(s_2)$
s_2	$0+\gamma v_{\pi_k}(s_1)$	$1+\gamma v_{\pi_k}(s_2)$	$-1+\gamma v_{\pi_k}(s_2)$

代入 $v_{\pi_0}(s_1)=-10, v_{\pi_0}(s_2)=-9$ 和 $\gamma=0.9$，得

$q_{\pi_0}(s,a)$	a_ℓ	a_0	a_r
s_1	-10	-9	-7.1
s_2	-9	-7.1	-9.1

选取 q_{π_0} 最大值对应的动作，改进后的新策略为：

$$\pi_1(a_r|s_1)=1, \quad \pi_1(a_0|s_2)=1.$$

仅需一次迭代即达到最优策略！在编程实现中，应持续迭代直到满足停止条件。

策略迭代算法——简单示例

▷ 第 $k=0$ 次迭代：步骤二：策略改进
$q_{\pi_k}(s,a)$ 的表达式：

$q_{\pi_k}(s,a)$	a_ℓ	a_0	a_r
s_1	$-1+\gamma v_{\pi_k}(s_1)$	$0+\gamma v_{\pi_k}(s_1)$	$1+\gamma v_{\pi_k}(s_2)$
s_2	$0+\gamma v_{\pi_k}(s_1)$	$1+\gamma v_{\pi_k}(s_2)$	$-1+\gamma v_{\pi_k}(s_2)$

代入 $v_{\pi_0}(s_1)=-10, v_{\pi_0}(s_2)=-9$ 和 $\gamma=0.9$，得

$q_{\pi_0}(s,a)$	a_ℓ	a_0	a_r
s_1	-10	-9	-7.1
s_2	-9	-7.1	-9.1

选取 q_{π_0} 最大值对应的动作，改进后的新策略为：

$$\pi_1(a_r|s_1)=1, \quad \pi_1(a_0|s_2)=1.$$

仅需一次迭代即达到最优策略！在编程实现中，应持续迭代直到满足停止条件。

策略迭代算法——简单示例

练习：将左侧格设为目标区域。

- 现在我们又掌握了一种搜索最优策略的强大算法！
- 接下来应用它，看看能发现什么。

策略迭代算法——简单示例

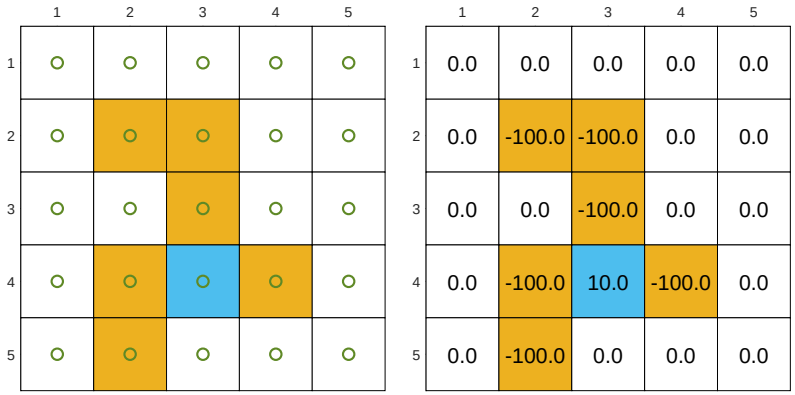
练习：将左侧格设为目标区域。

- 现在我们又掌握了一种搜索最优策略的强大算法！
- 接下来应用它，看看能发现什么。

策略迭代算法——复杂示例

▷ 设置：$r_{\text{boundary}} = -1$，$r_{\text{forbidden}} = -10$，$r_{\text{target}} = 1$，$\gamma = 0.9$。

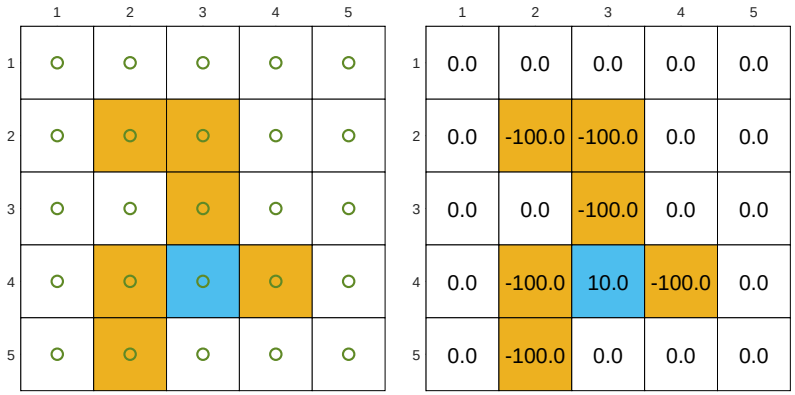
▷ 观察中间步骤的策略和状态价值。



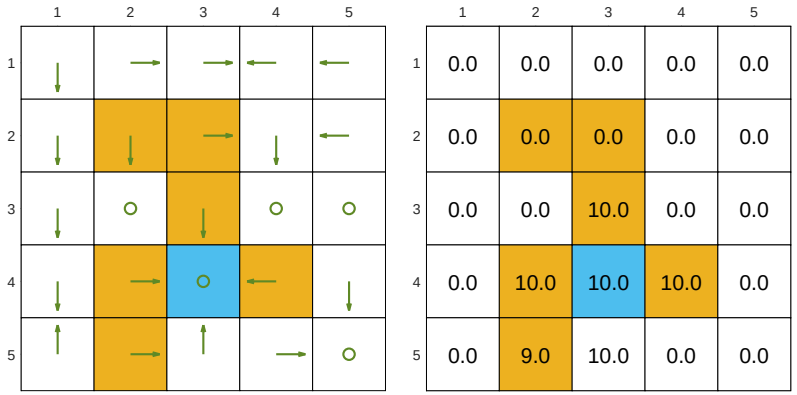
π_0 与 v_{π_0}

策略迭代算法——复杂示例

- 设置：$r_{\text{boundary}} = -1$，$r_{\text{forbidden}} = -10$，$r_{\text{target}} = 1$，$\gamma = 0.9$。
- 观察中间步骤的策略和状态价值。



π_0 与 v_{π_0}

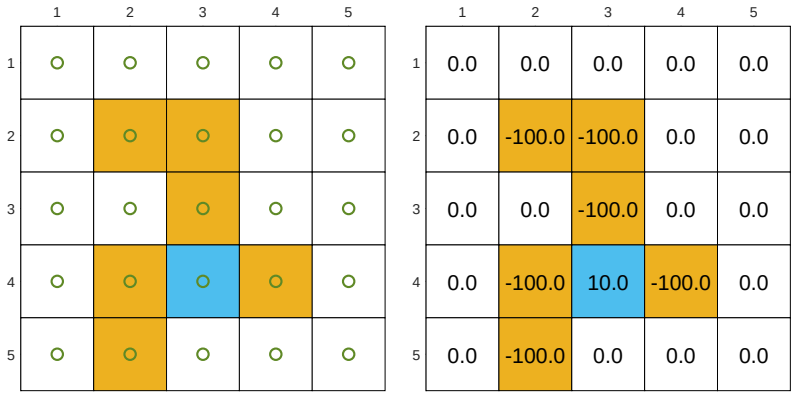


π_1 与 v_{π_1}

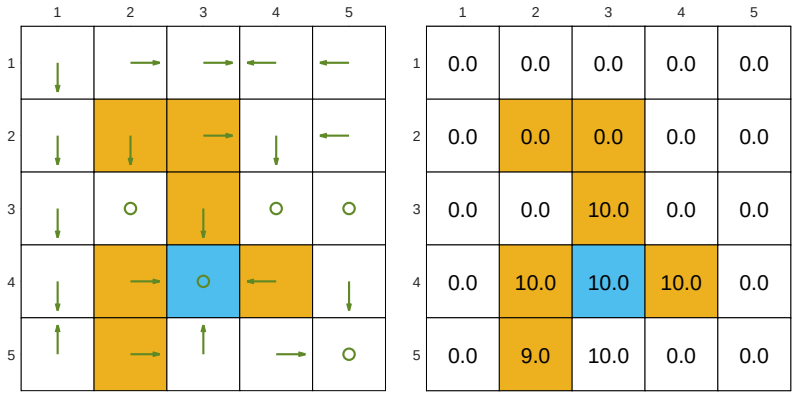
策略迭代算法——复杂示例

▷ 设置：$r_{\text{boundary}} = -1$，$r_{\text{forbidden}} = -10$，$r_{\text{target}} = 1$，$\gamma = 0.9$。

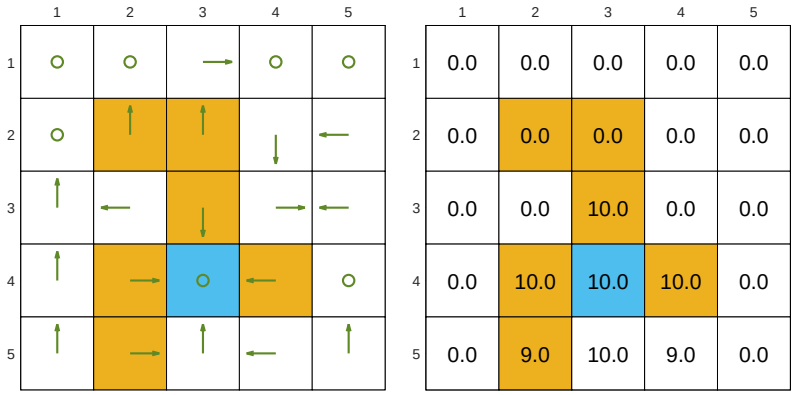
▷ 观察中间步骤的策略和状态价值。



π_0 与 v_{π_0}



π_1 与 v_{π_1}

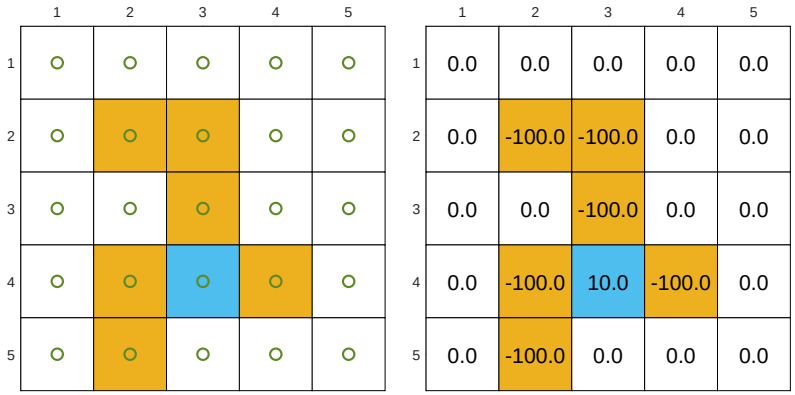


π_2 与 v_{π_2}

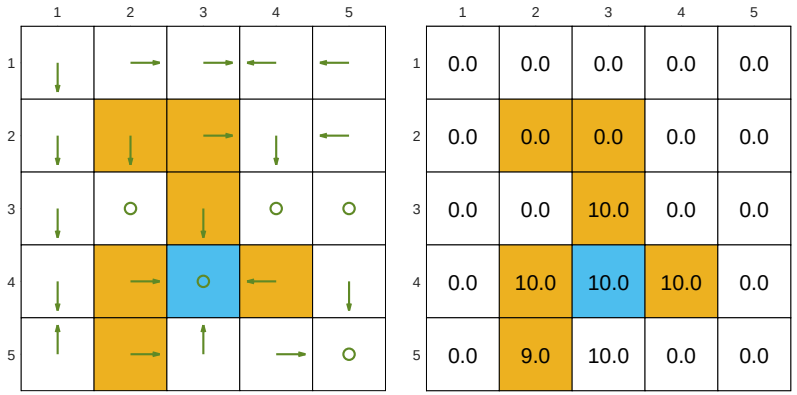
策略迭代算法——复杂示例

▷ 设置：$r_{\text{boundary}} = -1$，$r_{\text{forbidden}} = -10$，$r_{\text{target}} = 1$，$\gamma = 0.9$。

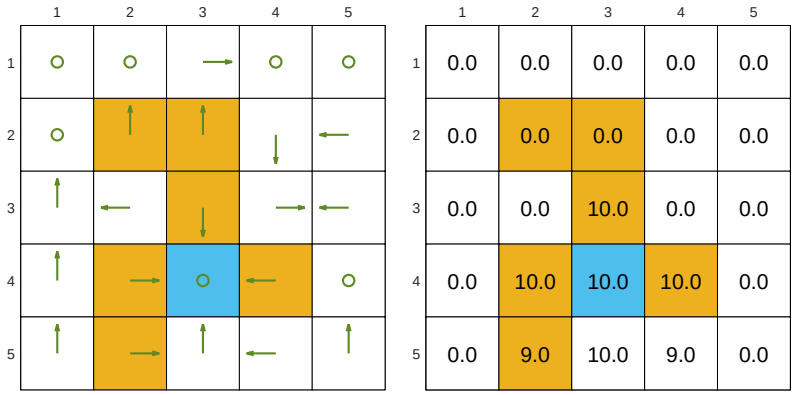
▷ 观察中间步骤的策略和状态价值。



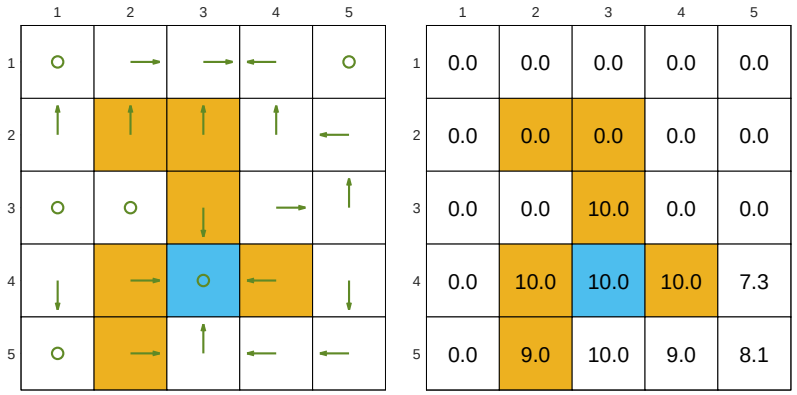
π_0 与 v_{π_0}



π_1 与 v_{π_1}



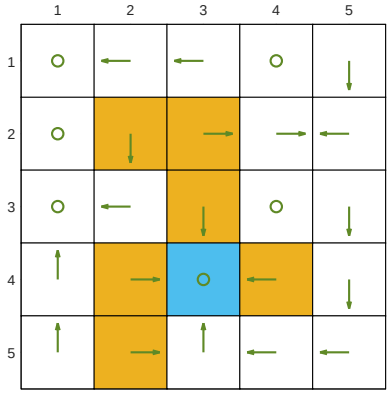
π_2 与 v_{π_2}



π_3 与 v_{π_3}

策略迭代算法——复杂示例

▷ 策略和状态价值的有趣演变规律

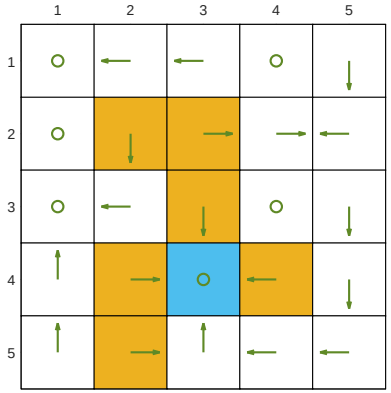


	1	2	3	4	5
1	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	10.0	0.0	6.6
4	0.0	10.0	10.0	10.0	7.3
5	0.0	9.0	10.0	9.0	8.1

π_4 与 v_{π_4}

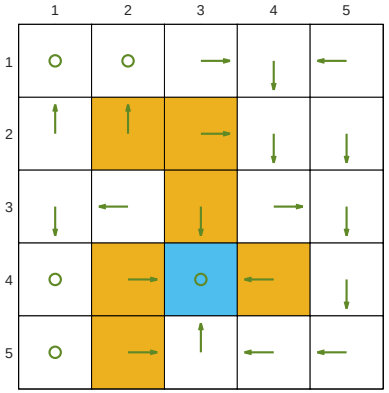
策略迭代算法——复杂示例

▷ 策略和状态价值的有趣演变规律



	1	2	3	4	5
1	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	10.0	0.0	6.6
4	0.0	10.0	10.0	10.0	7.3
5	0.0	9.0	10.0	9.0	8.1

π_4 与 v_{π_4}

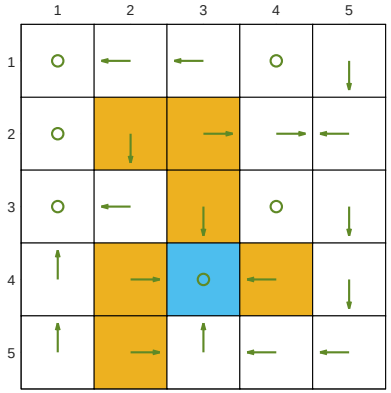


	1	2	3	4	5
1	0.0	0.0	4.3	4.8	4.3
2	0.0	0.0	4.8	5.3	5.9
3	0.0	0.0	10.0	5.9	6.6
4	0.0	10.0	10.0	10.0	7.3
5	0.0	9.0	10.0	9.0	8.1

π_5 与 v_{π_5}

策略迭代算法——复杂示例

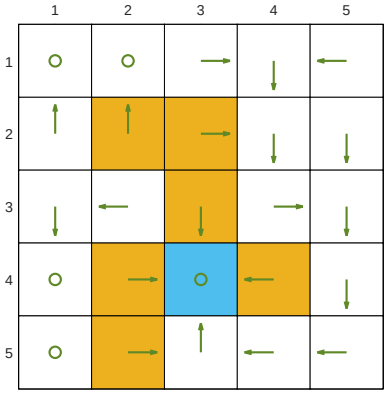
▷ 策略和状态价值的有趣演变规律



	1	2	3	4	5
1	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	10.0	0.0	6.6
4	0.0	10.0	10.0	10.0	7.3
5	0.0	9.0	10.0	9.0	8.1

π_4 与 v_{π_4}

$\vdots$



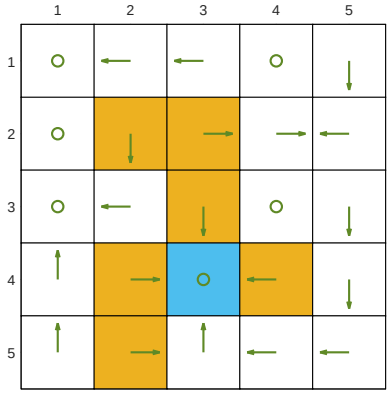
	1	2	3	4	5
1	0.0	0.0	4.3	4.8	4.3
2	0.0	0.0	4.8	5.3	5.9
3	0.0	0.0	10.0	5.9	6.6
4	0.0	10.0	10.0	10.0	7.3
5	0.0	9.0	10.0	9.0	8.1

π_5 与 v_{π_5}

$\vdots$

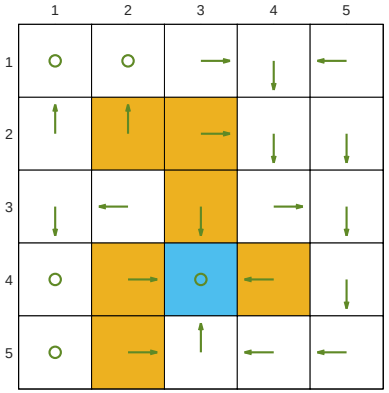
策略迭代算法——复杂示例

▷ 策略和状态价值的有趣演变规律



	1	2	3	4	5
1	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	10.0	0.0	6.6
4	0.0	10.0	10.0	10.0	7.3
5	0.0	9.0	10.0	9.0	8.1

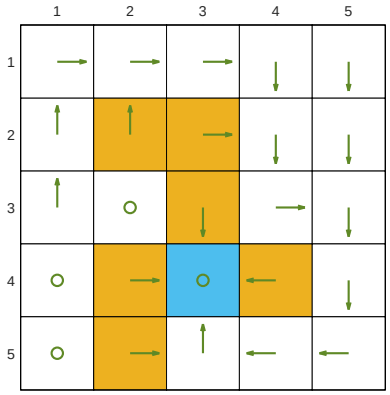
π_4 与 v_{π_4}



	1	2	3	4	5
1	0.0	0.0	4.3	4.8	4.3
2	0.0	0.0	4.8	5.3	5.9
3	0.0	0.0	10.0	5.9	6.6
4	0.0	10.0	10.0	10.0	7.3
5	0.0	9.0	10.0	9.0	8.1

π_5 与 v_{π_5}

⋮

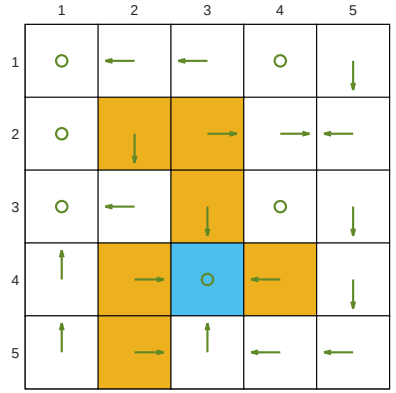


	1	2	3	4	5
1	3.5	3.9	4.3	4.8	5.3
2	3.1	3.5	4.8	5.3	5.9
3	2.8	0.0	10.0	5.9	6.6
4	0.0	10.0	10.0	10.0	7.3
5	0.0	9.0	10.0	9.0	8.1

π_9 与 v_{π_9}

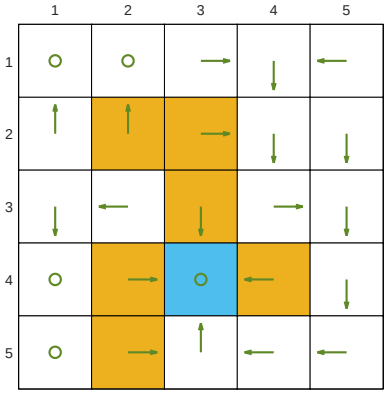
策略迭代算法——复杂示例

▷ 策略和状态价值的有趣演变规律



	1	2	3	4	5
1	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	10.0	0.0	6.6
4	0.0	10.0	10.0	10.0	7.3
5	0.0	9.0	10.0	9.0	8.1

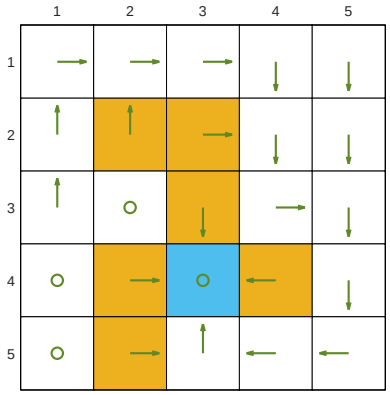
π_4 与 v_{π_4}



	1	2	3	4	5
1	0.0	0.0	4.3	4.8	4.3
2	0.0	0.0	4.8	5.3	5.9
3	0.0	0.0	10.0	5.9	6.6
4	0.0	10.0	10.0	10.0	7.3
5	0.0	9.0	10.0	9.0	8.1

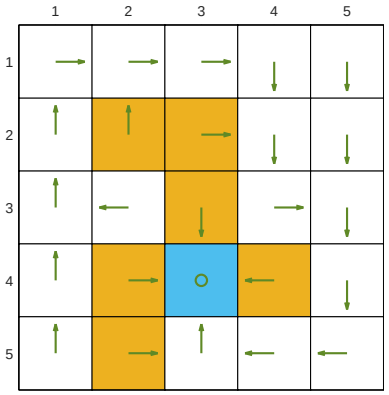
π_5 与 v_{π_5}

⋮



	1	2	3	4	5
1	3.5	3.9	4.3	4.8	5.3
2	3.1	3.5	4.8	5.3	5.9
3	2.8	0.0	10.0	5.9	6.6
4	0.0	10.0	10.0	10.0	7.3
5	0.0	9.0	10.0	9.0	8.1

π_9 与 v_{π_9}



	1	2	3	4	5
1	3.5	3.9	4.3	4.8	5.3
2	3.1	3.5	4.8	5.3	5.9
3	2.8	2.5	10.0	5.9	6.6
4	2.5	10.0	10.0	10.0	7.3
5	2.3	9.0	10.0	9.0	8.1

π_{10} 与 $v_{\pi_{10}}$

大纲

1 值迭代算法

2 策略迭代算法

3 截断策略迭代算法

值迭代与策略迭代的比较

策略迭代： 从 π_0 出发

- 策略评估（PE）：

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

- 策略改进（PI）：

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_{\pi_k})$$

值迭代： 从 v_0 出发

- 策略更新（PU）：

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_k)$$

- 价值更新（VU）：

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$$

值迭代与策略迭代的比较

策略迭代： 从 π_0 出发

- 策略评估（PE）：

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

- 策略改进（PI）：

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_{\pi_k})$$

值迭代： 从 v_0 出发

- 策略更新（PU）：

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_k)$$

- 价值更新（VU）：

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$$

值迭代与策略迭代的比较

▷ 两种算法非常相似:

$$\text{策略迭代:}\quad \pi_0 \xrightarrow{PE} v_{\pi_0} \xrightarrow{PI} \pi_1 \xrightarrow{PE} v_{\pi_1} \xrightarrow{PI} \pi_2 \xrightarrow{PE} v_{\pi_2} \xrightarrow{PI} \dots$$

$$\text{值迭代:}\quad u_0 \xrightarrow{PU} \pi_1' \xrightarrow{VU} u_1 \xrightarrow{PU} \pi_2' \xrightarrow{VU} u_2 \xrightarrow{PU} \dots$$

- PE=策略评估，PI=策略改进。
- PU=策略更新，VU=价值更新。

值迭代与策略迭代的比较

▷ 仔细比较各步骤:

	策略迭代算法	值迭代算法	说明
1) 策略:	π_0	无	
2) 价值:	$v_{\pi_0} = r_{\pi_0} + \gamma P_{\pi_0} v_{\pi_0}$	$v_0 \doteq v_{\pi_0}$	
3) 策略:	$\pi_1 = \arg\max_\pi (r_\pi + \gamma P_\pi v_{\pi_0})$	$\pi_1 = \arg\max_\pi (r_\pi + \gamma P_\pi v_0)$	两者策略相同
4) 价值:	$v_{\pi_1} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}$	$v_1 = r_{\pi_1} + \gamma P_{\pi_1} v_0$	由于 $v_{\pi_1} \geq v_{\pi_0}$，故 $v_{\pi_1} \geq v_1$
5) 策略:	$\pi_2 = \arg\max_\pi (r_\pi + \gamma P_\pi v_{\pi_1})$	$\pi_2' = \arg\max_\pi (r_\pi + \gamma P_\pi v_1)$	
⋮	⋮	⋮	⋮

说明:

- 两者从相同初始条件出发。
- 前三步相同。
- 第四步出现差异:
  - 策略迭代中，求解 $v_{\pi_1} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}$ 需要迭代算法（无限次迭代）。
  - 值迭代中，$v_1 = r_{\pi_1} + \gamma P_{\pi_1} v_0$ 仅需一步迭代。

值迭代与策略迭代的比较

▷ 仔细比较各步骤:

	策略迭代算法	值迭代算法	说明
1) 策略:	π_0	无	
2) 价值:	$v_{\pi_0} = r_{\pi_0} + \gamma P_{\pi_0} v_{\pi_0}$	$v_0 \doteq v_{\pi_0}$	
3) 策略:	$\pi_1 = \arg\max_\pi (r_\pi + \gamma P_\pi v_{\pi_0})$	$\pi_1 = \arg\max_\pi (r_\pi + \gamma P_\pi v_0)$	两者策略相同
4) 价值:	$v_{\pi_1} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}$	$v_1 = r_{\pi_1} + \gamma P_{\pi_1} v_0$	由于 $v_{\pi_1} \geq v_{\pi_0}$，故 $v_{\pi_1} \geq v_1$
5) 策略:	$\pi_2 = \arg\max_\pi (r_\pi + \gamma P_\pi v_{\pi_1})$	$\pi_2' = \arg\max_\pi (r_\pi + \gamma P_\pi v_1)$	
⋮	⋮	⋮	⋮

说明:

- 两者从相同初始条件出发。
- 前三步相同。
- 第四步出现差异:
  - 策略迭代中，求解 $v_{\pi_1} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}$ 需要迭代算法（无限次迭代）。
  - 值迭代中，$v_1 = r_{\pi_1} + \gamma P_{\pi_1} v_0$ 仅需一步迭代。

值迭代与策略迭代的比较

更具体地，分析求解 $v_{\pi_1} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}$ 的步骤:

$$
\begin{aligned}
v_{\pi_1}^{(0)} &= v_0 \\
v_{\pi_1}^{(1)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(0)} \\
v_{\pi_1}^{(2)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(1)} \\
&\vdots \\
v_{\pi_1}^{(j)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(j-1)} \\
&\vdots \\
v_{\pi_1}^{(\infty)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(\infty)}
\end{aligned}
$$

值迭代与策略迭代的比较

更具体地，分析求解 $v_{\pi_1} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}$ 的步骤:

$$
\begin{aligned}
& v_{\pi_1}^{(0)} = v_0 \\
\text{值迭代} \leftarrow v_1 \longleftarrow & v_{\pi_1}^{(1)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(0)} \\
& v_{\pi_1}^{(2)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(1)} \\
& \vdots \\
& v_{\pi_1}^{(j)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(j-1)} \\
& \vdots \\
& v_{\pi_1}^{(\infty)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(\infty)}
\end{aligned}
$$

值迭代与策略迭代的比较

更具体地，分析求解 $v_{\pi_1} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}$ 的步骤:

$$
\begin{aligned}
& v_{\pi_1}^{(0)} = v_0 \\
\text{值迭代} \leftarrow v_1 \longleftarrow & v_{\pi_1}^{(1)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(0)} \\
& v_{\pi_1}^{(2)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(1)} \\
& \vdots \\
& v_{\pi_1}^{(j)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(j-1)} \\
& \vdots \\
\text{策略迭代} \leftarrow v_{\pi_1} \longleftarrow & v_{\pi_1}^{(\infty)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(\infty)}
\end{aligned}
$$

值迭代与策略迭代的比较

更具体地，分析求解 $v_{\pi_1} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}$ 的步骤:

$$
\begin{aligned}
& & v_{\pi_1}^{(0)} &= v_0 \\
\text{值迭代} \leftarrow v_1 & \longleftarrow & v_{\pi_1}^{(1)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(0)} \\
& & v_{\pi_1}^{(2)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(1)} \\
& & &\vdots \\
\text{截断策略迭代} \leftarrow \bar{v}_1 & \longleftarrow & v_{\pi_1}^{(j)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(j-1)} \\
& & &\vdots \\
\text{策略迭代} \leftarrow v_{\pi_1} & \longleftarrow & v_{\pi_1}^{(\infty)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(\infty)}
\end{aligned}
$$

值迭代与策略迭代的比较

更具体地，分析求解 $v_{\pi_1} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}$ 的步骤:

$$
\begin{aligned}
& & v_{\pi_1}^{(0)} &= v_0 \\
\text{值迭代} \leftarrow v_1 & \longleftarrow & v_{\pi_1}^{(1)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(0)} \\
& & v_{\pi_1}^{(2)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(1)} \\
& & &\vdots \\
\text{截断策略迭代} \leftarrow \bar{v}_1 & \longleftarrow & v_{\pi_1}^{(j)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(j-1)} \\
& & &\vdots \\
\text{策略迭代} \leftarrow v_{\pi_1} & \longleftarrow & v_{\pi_1}^{(\infty)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(\infty)}
\end{aligned}
$$

- 值迭代算法计算一次。
- 策略迭代算法计算无限次。
- 截断策略迭代算法计算有限次（设为 j 次），从 j 到 ∞ 的迭代被截断。

截断策略迭代——伪代码

伪代码：截断策略迭代算法

初始化：所有 (s,a) 的概率模型 $p(r|s,a)$ 和 $p(s'|s,a)$ 已知。初始策略 π_0。

目标：搜索最优状态价值和最优策略。

当 v_k 尚未收敛时，执行第 k 次迭代：

策略评估：

初始化：选取初始值 $v_k^{(0)} = v_{k-1}$，最大迭代次数设为 j_{truncate}。

当 $j < j_{\text{truncate}}$ 时，执行：

对每个状态 $s \in \mathcal{S}$，执行：

$$v_k^{(j+1)}(s) = \sum_a \pi_k(a|s) \left[\sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_k^{(j)}(s')\right]$$

令 $v_k = v_k^{(j_{\text{truncate}})}$

策略改进：

对每个状态 $s \in \mathcal{S}$，执行：

对每个动作 $a \in \mathcal{A}(s)$，执行：

$$q_k(s,a) = \sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_k(s')$$

$a_k^*(s) = \arg\max_a q_k(s,a)$

若 $a = a_k^*$ 则 $\pi_{k+1}(a|s) = 1$，否则 $\pi_{k+1}(a|s) = 0$

截断策略迭代——收敛性

▷ 截断操作会影响收敛性吗?

Proposition (价值改进)

考虑用于策略评估步骤的迭代算法:

$$v_{\pi_k}^{(j+1)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(j)}, \quad j = 0, 1, 2, ...$$

若初始值选取为 $v_{\pi_k}^{(0)} = v_{\pi_{k-1}}$，则对每个 $j = 0, 1, 2, \ldots$，有

$$v_{\pi_k}^{(j+1)} \geq v_{\pi_k}^{(j)}$$

证明详见教材。

截断策略迭代——收敛性

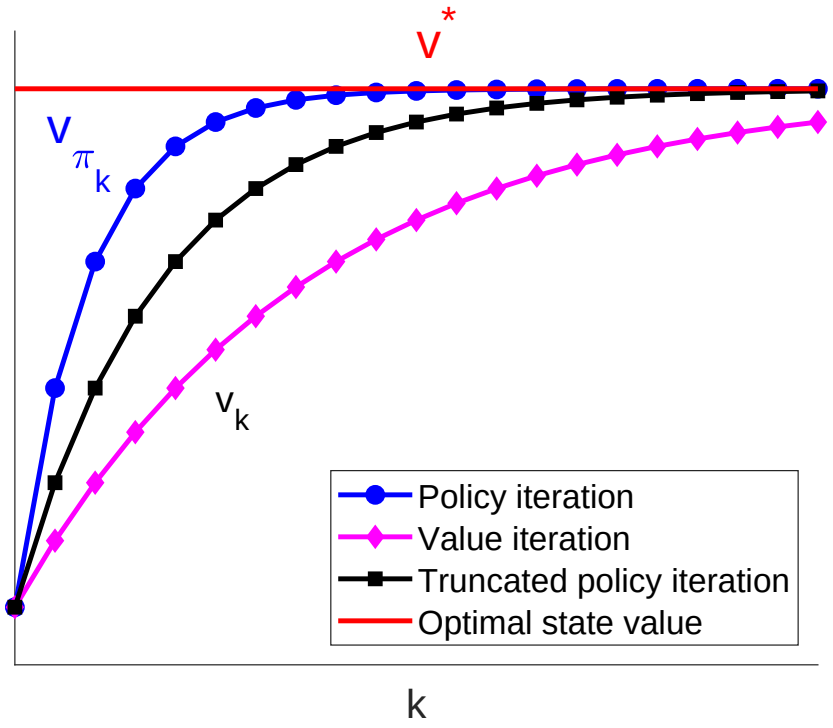


Figure: 值迭代、策略迭代与截断策略迭代关系示意图。

策略迭代的收敛性证明基于值迭代的收敛性。由于值迭代收敛，故策略迭代也收敛。

总结

▷ 值迭代：求解贝尔曼最优方程的迭代算法，给定初始值 v_0，

$$v_{k+1} = \max_\pi (r_\pi + \gamma P_\pi v_k)$$
$$\Updownarrow$$
$$\begin{cases} \text{策略更新：} \ \pi_{k+1} = \arg\max_\pi (r_\pi + \gamma P_\pi v_k) \\ \text{价值更新：} \ v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k \end{cases}$$

▷ 策略迭代：给定初始策略 π_0，

$$\begin{cases} \text{策略评估：} \ v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k} \\ \text{策略改进：} \ \pi_{k+1} = \arg\max_\pi (r_\pi + \gamma P_\pi v_{\pi_k}) \end{cases}$$

▷ 截断策略迭代